

Exploring Data Cleanness in Defects4J and Its Influence on Fault Localization Efficiency

Md Nakhla Rafi

Software Performance, Analysis, and
Reliability (SPEAR) Lab
Montréal, Québec, Canada
r_mdnaikh@encs.concordia.ca

Tse-Hsun (Peter) Chen

Software Performance, Analysis, and
Reliability (SPEAR) Lab
Montréal, Québec, Canada
peterc@encs.concordia.ca

An Ran Chen

University of Alberta
Edmonton, Alberta, Canada
anran6@ualberta.ca

Shaohua Wang

Central University of Finance and Economics
Beijing, China
davidshwang@ieee.org

ABSTRACT

Defects4J stands out as the most popular benchmark in software testing research, known for its comprehensive collection of real bugs from open-source systems. This paper presents an in-depth study of Defects4J's fault-triggering tests, particularly examining the influence of developer modifications post-bug reports on spectrum-based fault localization (SBFL) techniques. Our findings reveal that 55% of these tests were newly added and 22% modified with developer knowledge, impacting the accuracy of SBFL. Notably, SBFL techniques' performance drops significantly (up to -415% in Mean First Rank) when developer knowledge is absent in the tests. We provide a curated dataset of bugs without this knowledge, facilitating more realistic evaluations of SBFL techniques using Defects4J. This research offers insights for the development of future bug benchmarks.

1 INTRODUCTION

The complexity of modern software makes bug detection and fixing a challenging task. Developers often rely on techniques like spectrum-based fault localization (SBFL) and automated program repair for these tasks. SBFL techniques, as explored in [8, 10], prioritize potential bug locations by analyzing test coverage data, focusing on code segments more frequently covered by failing tests. Defects4J [1] has gained widespread popularity as a benchmark, offering real-world system data that is essential for evaluating various fault localization testing techniques [10].

However, concerns regarding the accuracy of fault localization methods have been highlighted due to the presence of developer knowledge in Defects4J's tests, particularly those derived from post-bug-fix versions [5, 6, 9]. These tests, often modified after addressing bugs, could inadvertently reveal the bug's location, thus potentially

Table 1: An excerpt of the Bug report CLI-51

BugID	CLI-51
Summary	Parameter value "something" misinterpreted as a parameter
Developer's comment	"Fix so parser doesn't burst options which are not defined. (-s) in the above case. Includes unit test [BugCLI51Test]."

skewing the results. While previous studies have indicated this issue [5, 9], they lack a comprehensive analysis of its prevalence and impact. The study investigates the role of developer knowledge in influencing fault-triggering tests within Defects4J, focusing on tests designed to fail when encountering bugs.

Our study utilized Defects4J v2.0.0 [1], a comprehensive dataset containing 835 bugs across 17 Java open-source systems. In this dataset, each bug is associated with a unique *fault-triggering test*, generally crafted by developers post-bug-fix, to replicate the bug consistently. An example of this is CLI-51 from Commons-CLI, highlighted in Table 1, where the associated test, was added after the bug was addressed, reflecting developer insights into the bug. The focus of this paper is to systematically explore how such tests in Defects4J, specifically those added or altered after bug fixes, affect the performance and reliability of fault localization techniques.

This study finds that 77% of Defects4J's fault-triggering tests embed developer knowledge. We identify five test modification categories, offering insights into bug-fixing and benchmark enhancement. Our analysis shows marked distortions in SBFL results due to this knowledge, with significant metric degradation. Our dataset, at [2], facilitates future research in fault localization.

2 STUDY DESIGN AND RESULTS

We conducted experiments on 809 bugs from 16 Java-based systems in Defects4J, excluding the *Chart* system due to its non-Git version control, which is crucial for our analysis of changes in fault-triggering tests. The number of tests triggering faults exceeds the bug count because some bugs are linked to multiple tests.

RQ1: Were Fault-triggering Tests Added/Modified After a Bug Was Reported? SBFL techniques' effectiveness may be influenced by developer knowledge in Defects4J tests, modified after bug reports, impacting fault localization reliability [3]. Using Jira, GitHub REST APIs, and git log, our study traced test modifications from bug

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE-Companion '24, April 14–20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0502-1/24/04...\$15.00
<https://doi.org/10.1145/3639478.3643125>

report to resolution in Defects4J, identifying alteration patterns and assessing the inclusion of developer knowledge post-bug reports.

We identified four patterns in Defects4J's fault-triggering tests: **Pattern 1 (53%)** includes tests created post-bug report, often as regression tests; **Pattern 2 (2%)** involves tests created and modified post-report with explicit bug knowledge; **Pattern 3 (22%)** covers existing tests modified post-report for better bug replication; and **Pattern 4 (23%)** encompasses tests that remain unmodified, indicating initial adequacy for bug detection. *Our study reveals that developer knowledge post-bug reporting affects 77% of tests in Defects4J, influencing their effectiveness in bug detection.*

RQ2: How Do Developers Modify the Fault-triggering Tests? When addressing bug fixes, developers often use test failures for insights into bugs' root causes [4, 7]. However, as RQ1 shows, many fault-triggering tests are either created or modified during bug fixing. Our study involved a manual analysis of 300 such tests, conducted in three phases. **Phase 1** entailed initial categorization by the first author, refined by the second author. **Phase 2** included an independent review and categorization by both authors. **Phase 3** resolved disagreements, achieving high consensus (Cohen's Kappa of 0.86). This process revealed that test modifications primarily focus on improving test oracles or updating tests due to source code changes.

We then categorized the modifications developers make to fault-triggering tests. The main categories include: **Adding New Tests (63%)**, where tests are created to replicate bugs; **Test Co-evolution (19%)**, modifying tests to align with source code changes; **Adding New Assertion (14%)**, where new assertions are added to existing tests to better capture bugs; **Improving Test Logic During Bug Fixes (2%)**, altering test logic for better bug replication; and **Others (1%)**, like code formatting or cleanup. This analysis explores how developers modify tests for replicating and fixing bugs.

RQ3: How does Having Developer Knowledge in Fault-triggering Tests Affect Fault Localization Results? RQ3 investigates how developer knowledge in fault-triggering tests affects fault localization results. We first focus on bugs whose tests existed before the bug report but were modified afterward (Pattern 3 from RQ2). The study compares the performance of spectrum-based fault localization (SBFL) techniques on two versions: the original buggy version *vBuggy* and the Defects4J version *vD4J* where tests were modified. Data for *vBuggy* was collected through extensive manual effort, including configuring JaCoCo for test coverage. The effectiveness of SBFL is assessed using metrics like Top-K, Mean First Rank (MFR), and Mean Average Rank (MAR), providing insights into the impact of incorporating developer knowledge on fault localization.

Our research indicates notably poorer fault localization in vBuggy compared to vD4J for Pattern 3 bugs. In *vBuggy*, all techniques showed decreased performance, with low ranking of faulty elements. In *vBuggy*, MFR and MAR were around 3,000, far exceeding *vD4J*'s 200-1,200, demonstrating significant developer knowledge impact on fault localization. See Table 2 for details.

We also evaluated FL techniques on Pattern 4 bugs, where fault-triggering tests were unchanged. Surprisingly, the results were again poorer in *vBuggy* compared to *vD4J* for the 105 analyzed bugs. Rankings such as Top-1, Top-3, and Top-5 were lower in *vBuggy* (2 to 6) compared to *vD4J* (9 to 28), with MFR and MAR values indicating a 200% decrease in effectiveness in *vBuggy*.

Table 2: Fault localization results for Pattern 3 & 4 bugs

Pattern	Technique	Version	Bugs	Top-1	Top-3	Top-5	MFR	MAR
3	Ochiai	<i>vBuggy</i>	118	3	3	3	2,965 (-415%)	3,254 (-268%)
		<i>vD4J</i>	118	49	64	70	714	1,214
	Tarantula	<i>vBuggy</i>	118	4	4	4	2,988 (-424%)	3,244 (-262%)
		<i>vD4J</i>	118	47	62	66	705	1,238
	D-Star	<i>vBuggy</i>	118	10	11	15	2,827 (-104%)	3,231 (-701%)
		<i>vD4J</i>	118	29	39	44	273	461
4	Barinel	<i>vBuggy</i>	118	3	3	3	2,954 (-378%)	3,282 (-272%)
		<i>vD4J</i>	118	45	60	65	781	1,208
	Ochiai	<i>vBuggy</i>	105	2	2	6	2,097 (-245%)	2,561 (-203%)
		<i>vD4J</i>	105	10	18	26	855	1,264
	Tarantula	<i>vBuggy</i>	105	2	2	6	2,100 (-247%)	2,587 (-206%)
		<i>vD4J</i>	105	12	17	28	851	1,254
	D-Star	<i>vBuggy</i>	105	2	2	5	2,073 (-250%)	2,553 (-195%)
		<i>vD4J</i>	105	10	16	23	829	1,309
	Barinel	<i>vBuggy</i>	105	2	2	5	2,075 (-234%)	2,564 (-185%)
		<i>vD4J</i>	105	9	17	28	888	1,389

In both Pattern 3 and 4, about 85% of fault-triggering tests failed in *vBuggy*, but were ineffective in bug detection. Notably, in Pattern 4, despite unmodified tests, fault localization results in *vBuggy* decreased significantly. Manual analysis revealed that developer knowledge, introduced through changes in functions or test setups, affected these results. *This highlights the need for future studies to use unmodified fault-triggering tests post-bug reports for more realistic evaluations of fault localization techniques.*

3 CONCLUSION

In conclusion, our research underscores the influence of developer knowledge in Defects4J's fault-triggering tests on fault localization. Our findings suggest a need to reassess these techniques using our dataset, which is devoid of developer bias and stresses the need for benchmarks that mirror actual bug-fixing processes. Our dataset, available at [2], aids this and future research, enhancing benchmarks in software testing.

REFERENCES

- [1] 2022. The Defects4J dataset version 2.0.0. <https://github.com/rjust/defects4j>.
- [2] 2023. Public repository for double-anonymous submission. <https://github.com/double-anonymous-submission/ICSE-paper846>.
- [3] An Ran Chen, Tse-Hsun Chen, and Junjie Chen. 2022. How Useful is Code Change Information for Fault Localization in Continuous Integration?. In *37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [4] Bo Jiang, Zhenyu Zhang, Tsun-Him Tse, and Tsong Yueh Chen. 2009. How well do test case prioritization techniques support statistical fault localization. In *2009 33rd Annual IEEE International Computer Software and Applications Conference*, Vol. 1. IEEE, 99–106.
- [5] René Just, Chris Parnin, Ian Drosos, and Michael D Ernst. 2018. Comparing developer-provided to user-provided tests for fault localization and automated program repair. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 287–297.
- [6] Vinay Kabadi, Dezhen Kong, Siyu Xie, Lingfeng Bao, Gede Artha Azriadi Prana, Tien-Duy B Le, Xuan-Bach D Le, and David Lo. 2023. The Future Can't Help Fix The Past: Assessing Program Repair In The Wild. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 50–61.
- [7] Dongsun Kim, Yida Tao, Sunghun Kim, and Andreas Zeller. 2013. Where should we fix this bug? a two-phase recommendation model. *IEEE transactions on software Engineering* 39, 11 (2013), 1597–1610.
- [8] Kui Liu, Anil Koyuncu, Tegawendé F Bissyandé, Dongsun Kim, Jacques Klein, and Yves Le Traon. 2019. You cannot fix what you cannot find! an investigation of fault localization bias in benchmarking automated program repair systems. In *2019 12th IEEE conference on software testing, validation and verification (ICST)*. IEEE, 102–113.
- [9] Kui Liu, Li Li, Anil Koyuncu, Dongsun Kim, Zhe Liu, Jacques Klein, and Tegawendé F Bissyandé. 2021. A critical review on the evaluation of automated program repair systems. *Journal of Systems and Software* 171 (2021), 110817.
- [10] Jeongju Sohn and Shin Yoo. 2017. Flucss: Using code and change metrics to improve fault localization. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 273–283.